Another reason to support HTTPS is that many browser features are available only when your site is served over HTTPS. Some of these features are JavaScript browser APIs, such as location APIs, microphone APIs, and storage APIs. These are available only over HTTPS to protect users from attackers that could modify insecure HTTP requests. Other features apply to server-side apps too, such as Brotli compression and HTTP/2 support.

TIP For details on how the SSL/TLS protocols work, see chapter 9 of *Real-World Cryptography*, by David Wong (Manning, 2021), http://mng.bz/zxz1.

To enable HTTPS, you need to obtain and configure a TLS certificate for your server. Unfortunately, although that process is a lot easier than it used to be and is now essentially free thanks to Let's Encrypt (https://letsencrypt.org), it's still far from simple in many cases. If you're setting up a production server, I recommend carefully following the tutorials on the Let's Encrypt site. It's easy to get it wrong, so take your time.

TIP If you're hosting your app in the cloud, most providers will provide one-click TLS certificates so that you don't have to manage certificates yourself. This is extremely useful, and I highly recommend it for everyone. You don't even have to host your application in the cloud to take advantage of this. Cloudflare (https://www.cloudflare.com) provides a CDN service that you can add TLS to. You can even use it for free.

As an ASP.NET Core application developer, you can often get away without directly supporting HTTPS in your app by taking advantage of the reverse-proxy architecture, as shown in figure 28.3, in a process called SSL/TLS offloading/termination. This is generally standard in Platform as a Service (PaaS) cloud services, such as Azure App Service.